

Importing the Dependencies

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score
import sklearn
```

```
print("numpy" , np.__version__)
print("pandas" , pd.__version__)
print("sklearn", sklearn.__version__)
```

```
numpy 2.0.2
pandas 2.2.2
sklearn 1.6.1
```

Data Collection and Analysis

PIMA Diabetes DATASET

```
# loading the diabetes dataset to a Panda Dataframe

diabetes_dataset = pd.read_csv('/content/diabetes.csv')
```

```
# what does a function do?
```

```
pd.read_csv? # What does pd.read_csv() do?
```

```
File "/tmp/ipykernel\_5483/2685011641.py", line 3
  pd.read_csv? # What does pd.read_csv() do?
    ^
SyntaxError: invalid syntax
```

Next steps: [Explain error](#)

```
# printing the first five 5 rows of the dataset
diabetes_dataset.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPec
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

Next steps: [Generate code with diabetes_dataset](#) [New interactive sheet](#)

```
# Number of rows and columns in this dataset
diabetes_dataset.shape # Data is collected from 768 people and has 9 columns

(768, 9)
```

```
# getting the statistical measures of the data
diabetes_dataset.describe()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100

```
diabetes_dataset['Outcome'].value_counts()
```

```

      count
Outcome
0         500
1         268

```

dtype: int64

0 --> Non-Diabetic people 1 --> Diabetic people

```
diabetes_dataset.groupby('Outcome').mean()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	
Outcome						
0	3.298000	109.980000	68.184000	19.664000	68.792000	30.30
1	4.865672	141.257463	70.824627	22.164179	100.335821	35.14

```

# Separating the data and labels
X = diabetes_dataset.drop(columns = 'Outcome', axis=1)
Y = diabetes_dataset['Outcome']

```

```
print(X)
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1
..	...	...	...	...	...	...
763	10	101	76	48	180	32.9
764	2	122	70	27	0	36.8
765	5	121	72	23	112	26.2
766	1	126	60	0	0	30.1
767	1	93	70	31	0	30.4

	DiabetesPedigreeFunction	Age
0	0.627	50
1	0.351	31
2	0.672	32
3	0.167	21
4	2.288	33
..	...	...
763	0.171	63
764	0.340	27

```
765          0.245    30
766          0.349    47
767          0.315    23
```

```
[768 rows x 8 columns]
```

```
print(Y)
```

```
0      1
1      0
2      1
3      0
4      1
..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

Data Standardization

Data Standardization is used to make our ML model do the best predictions from the dataset

```
scaler = StandardScaler()
```

```
scaler.fit(X)
```

```
▼ StandardScaler ⓘ ?
StandardScaler()
```

```
standardized_data = scaler.transform(X)
```

```
print(standardized_data) # All values between 0 and 1, hence the data is standardized
```

```
[[ 0.63994726  0.84832379  0.14964075 ...  0.20401277  0.46849198
   1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
  -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
  -0.10558415]
 ...
 [ 0.3429808   0.00330087  0.14964075 ... -0.73518964 -0.68519336
  -0.27575966]
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101
   1.17073215]
```

```
[[-0.84488505 -0.8730192  0.04624525 ... -0.20212881 -0.47378505
 -0.87137393]]
```

```
X = standardized_data          # X->DATA
Y = diabetes_dataset['Outcome'] # Y->MODEL
```

```
print(X)
print(Y)
```

```
[[ 0.63994726  0.84832379  0.14964075 ...  0.20401277  0.46849198
  1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
 -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
 -0.10558415]
 ...
 [ 0.3429808  0.00330087  0.14964075 ... -0.73518964 -0.68519336
 -0.27575966]
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101
  1.17073215]
 [-0.84488505 -0.8730192  0.04624525 ... -0.20212881 -0.47378505
 -0.87137393]]
0      1
1      0
2      1
3      0
4      1
..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

Train Test Split

```
X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size = 0.2, strat
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
# total data items = 768
# data items used for training = 614
# data items used for testing = 154
```

```
(768, 8) (614, 8) (154, 8)
```

Training the model

```
classifier = svm.SVC(kernel='linear')
```

```
# training the support vector machine classifier
classifier.fit(X_train, Y_train)
```

▼ SVC ⓘ ?

SVC(kernel='linear')

Model Evaluaion

Accuracy Score

```
# accuracy score on the training data
X_train_prediction = classifier.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)
```

```
print('Accuracy score of the training data : ', training_data_accuracy) # data
```

```
Accuracy score of the training data :  0.7866449511400652
```

```
# accuracy score on the test data
X_test_prediction = classifier.predict(X_test)
test_data_accuracy = accuracy_score(X_test_prediction, Y_test)
```

```
print('Accuracy score of the test data : ', test_data_accuracy) # data accuracy
```

```
Accuracy score of the test data :  0.7727272727272727
```

Making a predictive system

```
input_data = (1,189,60,23,846,30.1,0.398,59)    # required input fields for prec

# changing the input_data to numpy array
input_data_as_numpy_array = np.asarray(input_data)

# reshape the array as we are predicting for one instance
input_data_reshaped = input_data_as_numpy_array.reshape(1, -1)

# standardize the input data
std_data = scaler.transform(input_data_reshaped)
print(std_data)

prediction = classifier.predict(std_data)
print(prediction)
```

```

if (prediction[0] == 0):
    print('The person is not diabetic')
else:
    print('The person is diabetic, get a pathology check')

[[-0.84488505  2.13150675 -0.47073225  0.15453319  6.65283938 -0.24020459
  -0.2231152   2.19178518]]
[1]
The person is diabetic, get a pathology check
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning:
  warnings.warn(

```

The O/P of our model will be a "list" not integer.

✓ What is a Pickle File?

A pickle file (`.pkl`) is a Python-specific way to serialize and deserialize Python object structures. This means you can convert a Python object (like your trained machine learning model or a `StandardScaler` object) into a byte stream, and then reconstruct it back into a Python object later. This is useful for saving the state of your model so you don't have to retrain it every time you want to use it.

```

import pickle

# Save the trained classifier model
filename = 'diabetes_model.pkl'
pickle.dump(classifier, open(filename, 'wb'))

# Save the scaler object
scaler_filename = 'scaler.pkl'
pickle.dump(scaler, open(scaler_filename, 'wb'))

print(f"Model saved to {filename}")
print(f"Scaler saved to {scaler_filename}")

Model saved to diabetes_model.pkl
Scaler saved to scaler.pkl

```

✓ Loading the saved model and scaler (for demonstration)

To use your saved model and scaler in a new session or script, you would load them back using `pickle.load()`:

```
# Load the saved model
loaded_model = pickle.load(open('diabetes_model.pkl', 'rb'))

# Load the saved scaler
loaded_scaler = pickle.load(open('scaler.pkl', 'rb'))

# Example of using the loaded model and scaler for prediction
# (using the same input data as before for consistency)

input_data = (1,189,60,23,846,30.1,0.398,59)
input_data_as_numpy_array = np.asarray(input_data)
input_data_resaped = input_data_as_numpy_array.reshape(1, -1)

# Standardize the input data using the loaded scaler
std_data_loaded = loaded_scaler.transform(input_data_resaped)

# Make prediction using the loaded model
prediction_loaded = loaded_model.predict(std_data_loaded)

if (prediction_loaded[0] == 0):
    print('The person is not diabetic (prediction from loaded model)')
else:
    print('The person is diabetic, get a pathology check (prediction from loaded model)')
The person is diabetic, get a pathology check (prediction from loaded model)
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning:
warnings.warn(
```

Start coding or [generate](#) with AI.